PROYECTO INTERMODULAR DAW

LIFEHUB

Un espacio creativo personal

Introducción

LifeHub es una aplicación web que permite a los usuarios disponer de un espacio creativo personal donde escribir, organizar ideas y compartir contenido de forma controlada.

Proyecto realizado por Gerónimo Molero Rodríguez 2ºDAW (2025-26)

Índice

1. Justificación del proyecto
    - Casos de uso del sistema
2. Resumen del proyecto
    - Alcance del proyecto
    - Contenidos del ciclo formativo integrados
    - Límites del proyecto
3. Objetivos y requisitos
    - Objetivo general
    - Objetivos específicos
    - Requisitos funcionales
    - Requisitos no funcionales
4. Arquitectura inicial del sistema
    - Componentes principales
    - Diagrama de red
    - Flujo de login con JWT
    - Modelo de datos
    - Arquitectura de contenedores
5. Planificación preliminar
    - Fases
    - Riesgos identificados
6. Preparación del entorno de trabajo
    - Estructura del proyecto
    - Control de versiones
7. Bitácora

1- Justificación del proyecto

El desarrollo de LifeHub surge de la necesidad de disponer de un entorno digital que facilite la creación, organización y consulta de contenido en un único espacio de trabajo. En muchos contextos, la producción de textos largos, documentación técnica o apuntes requiere utilizar múltiples herramientas diferentes, lo que provoca interrupciones constantes en el flujo de trabajo.

Las aplicaciones tradicionales de escritura suelen centrarse únicamente en el texto, obligando al usuario a alternar entre distintas plataformas para consultar referencias, reproducir contenido multimedia o almacenar recursos relacionados con el contenido que está desarrollando. Esta fragmentación puede generar cansancio, pérdida de concentración y menor eficiencia durante el proceso creativo.

LifeHub propone un enfoque diferente: ofrecer un espacio personal donde el usuario pueda escribir, organizar ideas y mantener referencias en el mismo entorno. La integración de contenido multimedia permite utilizar vídeos, música u otros recursos externos como apoyo durante la escritura o el estudio, reduciendo la necesidad de cambiar de aplicación continuamente.

El objetivo del proyecto es, por tanto, mejorar la experiencia de creación de contenido, fomentar la creatividad y facilitar la organización de información en un entorno digital unificado. Aunque en esta primera versión se plantea como un MVP funcional, la arquitectura del sistema está diseñada para permitir futuras ampliaciones y funcionalidades adicionales.

Casos de uso del sistema

A continuación se describen algunos escenarios representativos en los que LifeHub puede resultar especialmente útil:

Estudiantes que toman apuntes de clases grabadas

Un estudiante puede reproducir una clase grabada o un vídeo educativo mientras escribe apuntes directamente en el editor de documentos. La posibilidad de mantener el contenido multimedia y las notas en el mismo espacio facilita la concentración y permite organizar mejor la información.

Escritura creativa

Un escritor puede utilizar LifeHub como un espacio personal para desarrollar ideas, redactar textos y mantener versiones anteriores de sus documentos. El sistema de versionado permite revisar cambios o recuperar versiones previas del contenido si es necesario.

Redacción de documentación técnica

Un desarrollador o profesional puede utilizar la aplicación para redactar documentación en formato Markdown, manteniendo referencias externas como vídeos explicativos, tutoriales o recursos relacionados con el contenido que está elaborando.

Espacios personales de conocimiento

Los usuarios pueden utilizar LifeHub como un repositorio personal de ideas, notas y recursos, organizando su información en espacios creativos y compartiendo contenidos específicos con otros usuarios cuando sea necesario.

2- Resumen del proyecto

LifeHub es una aplicación web que permite a los usuarios disponer de un espacio creativo personal donde escribir, organizar ideas y compartir contenido de forma controlada.

El sistema permite:

- Crear espacios personales de escritura en Markdown.
- Gestionar recursos externos legales (embebidos como vídeos o música).
- Compartir contenidos con usuarios de confianza.
- Publicar una parte limitada del contenido en un perfil público.
- Mantener un historial de versiones de documentos, permitiendo recuperar versiones anteriores.

El proyecto se centra en el desarrollo de un MVP funcional, priorizando la estabilidad y seguridad del sistema.

Alcance del proyecto

El sistema incluirá:

- Registro e inicio de sesión de usuarios.
- Gestión de perfiles.
- Creación y edición de espacios creativos.
- Edición de documentos en Markdown.
- Guardado automático de versiones de documentos.
- Control de privacidad (privado / compartido).
- Visualización de recursos externos embebidos.
- Perfiles públicos limitados.
- Panel de administración básico para gestión del sistema.
- Registro de logs de actividad para auditoría básica.

Contenidos del ciclo formativo que se integran

Desarrollo web

- Frontend con Angular.
- Backend REST con .NET Web API.

Bases de datos

- Diseño de base de datos relacional.
- Modelado de entidades y relaciones.
- Gestión de historiales de versiones.

Despliegue de aplicaciones

- Contenedores Docker.
- Docker Compose para levantar todos los servicios del sistema.
- Arquitectura cliente-servidor.

Entornos de desarrollo

- Control de versiones con Git.
- Organización del proyecto y repositorio.

Seguridad

- Autenticación mediante JWT.
- Control de roles y permisos.
- Registro de actividad del sistema.

Límites del proyecto

Para mantener el alcance controlado, el MVP no incluirá inicialmente:

- Edición colaborativa en tiempo real.
- Sistema completo de mensajería.
- Gestión avanzada de recursos multimedia.
- Aplicación móvil.

Estas funcionalidades quedan planteadas como posibles ampliaciones futuras.

3- Objetivos y requisitos del proyecto

Objetivo general

Desarrollar una aplicación web segura que permita a los usuarios crear y gestionar espacios creativos personales con posibilidad de compartir contenido de forma controlada.

Objetivos específicos

- Implementar un sistema de autenticación seguro.
- Permitir la creación y edición de documentos Markdown.
- Gestionar espacios creativos individuales.
- Implementar un sistema de versionado de documentos.
- Permitir compartir contenidos con usuarios concretos.
- Implementar perfiles públicos con contenido limitado.
- Desarrollar un panel de administración básico.
- Registrar actividad relevante del sistema mediante logs.
- Desplegar la aplicación mediante contenedores Docker.

Requisitos funcionales

El sistema deberá permitir:

- Registro de usuarios.
- Inicio y cierre de sesión.
- Creación de espacios creativos.
- Edición de documentos en Markdown.
- Guardado de versiones del documento.
- Consulta y restauración de versiones anteriores.
- Compartir espacios con usuarios de confianza.
- Configurar la privacidad de los espacios.
- Visualizar recursos externos embebidos.
- Mostrar perfiles públicos con contenido limitado.
- Acceso a un panel de administración para gestionar usuarios y recursos.
- Registro de actividad relevante del sistema.

Requisitos no funcionales

- Autenticación segura mediante JWT.
- Arquitectura cliente-servidor REST.
- Base de datos relacional.
- Uso de contenedores Docker para el despliegue.
- Despliegue simplificado mediante Docker Compose.
- Control de errores y validación de datos.
- Interfaz web accesible desde navegadores modernos.
- Registro de logs para auditoría básica del sistema.

4- Arquitectura inicial del sistema

El sistema seguirá una arquitectura web de **tres capas.**

Componentes principales

Cliente (Frontend)

Aplicación desarrollada en Angular que se ejecuta en el navegador del usuario.

Funciones:

- Interfaz de usuario
- Edición Markdown
- Gestión de espacios
- Visualización de versiones de documentos
- Comunicación con la API

Servidor de aplicaciones (Backend)

Desarrollado con .NET Web API.

Responsabilidades:

- Lógica de negocio
- Autenticación y autorización
- Gestión de usuarios y espacios
- Gestión de versiones de documentos
- Registro de logs de actividad
- Comunicación con la base de datos

Base de datos

Base de datos relacional:

- SQL Server o PostgreSQL

Contendrá información sobre:

- Usuarios
- Espacios
- Documentos
- Versiones de documentos
- Permisos de acceso
- Logs de actividad

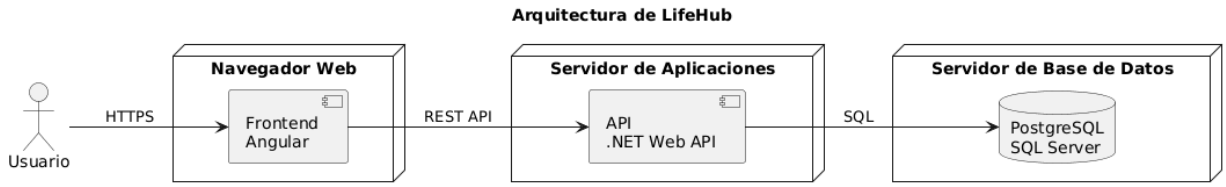
Diagrama de red simplificado

El usuario accede a la aplicación mediante el navegador web.

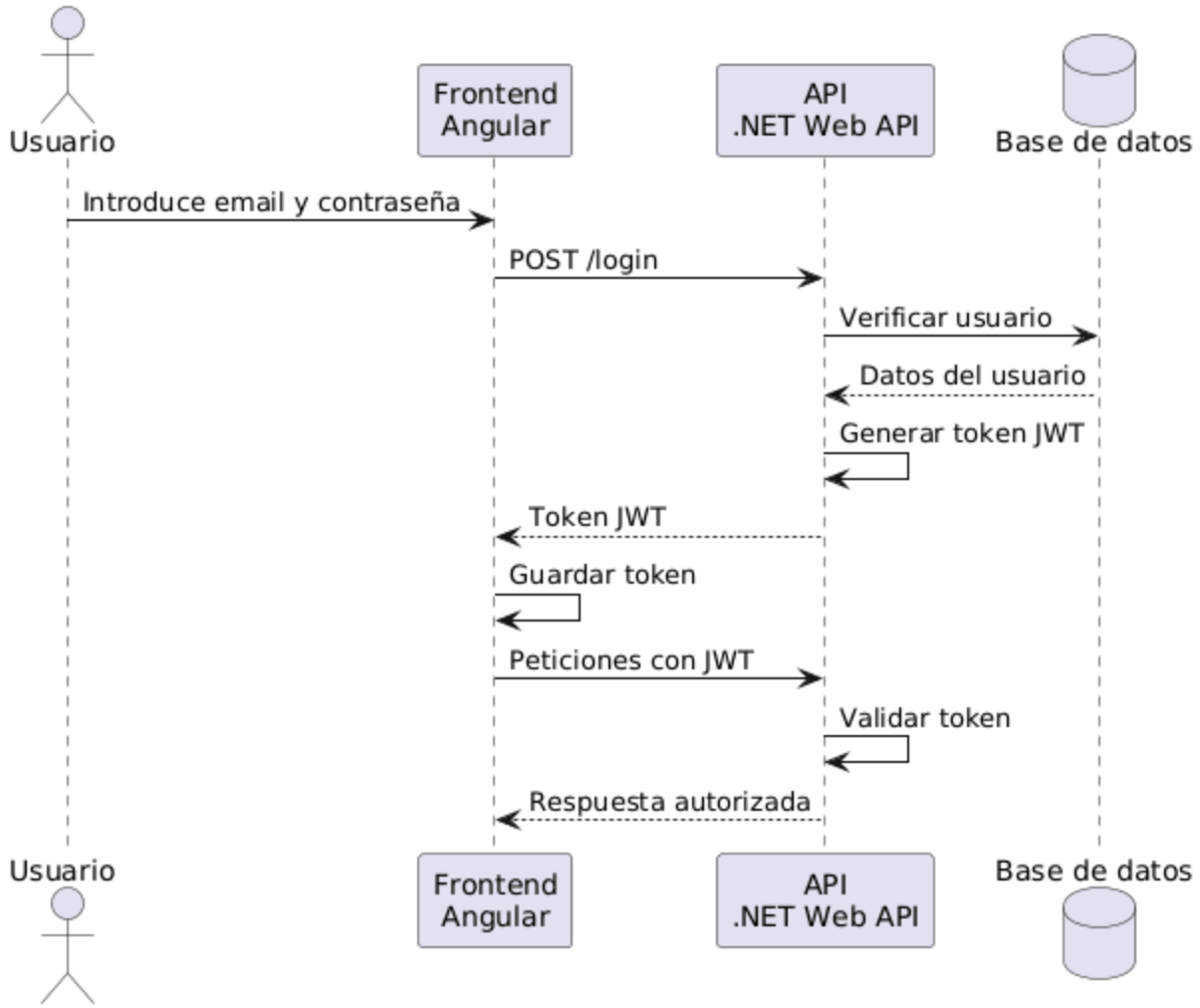
Las peticiones se envían al frontend Angular, que se comunica con la API REST mediante HTTP.

El backend procesa las solicitudes y accede a la base de datos relacional para almacenar o recuperar información.

Todo el sistema se ejecutará en contenedores Docker conectados mediante Docker Compose.



Flujo del login con JWT



Modelo de datos

Las entidades principales del sistema serán:

- Usuario
- EspacioCreativo
- Documento
- VersionDocumento
- Permiso
- LogActividad

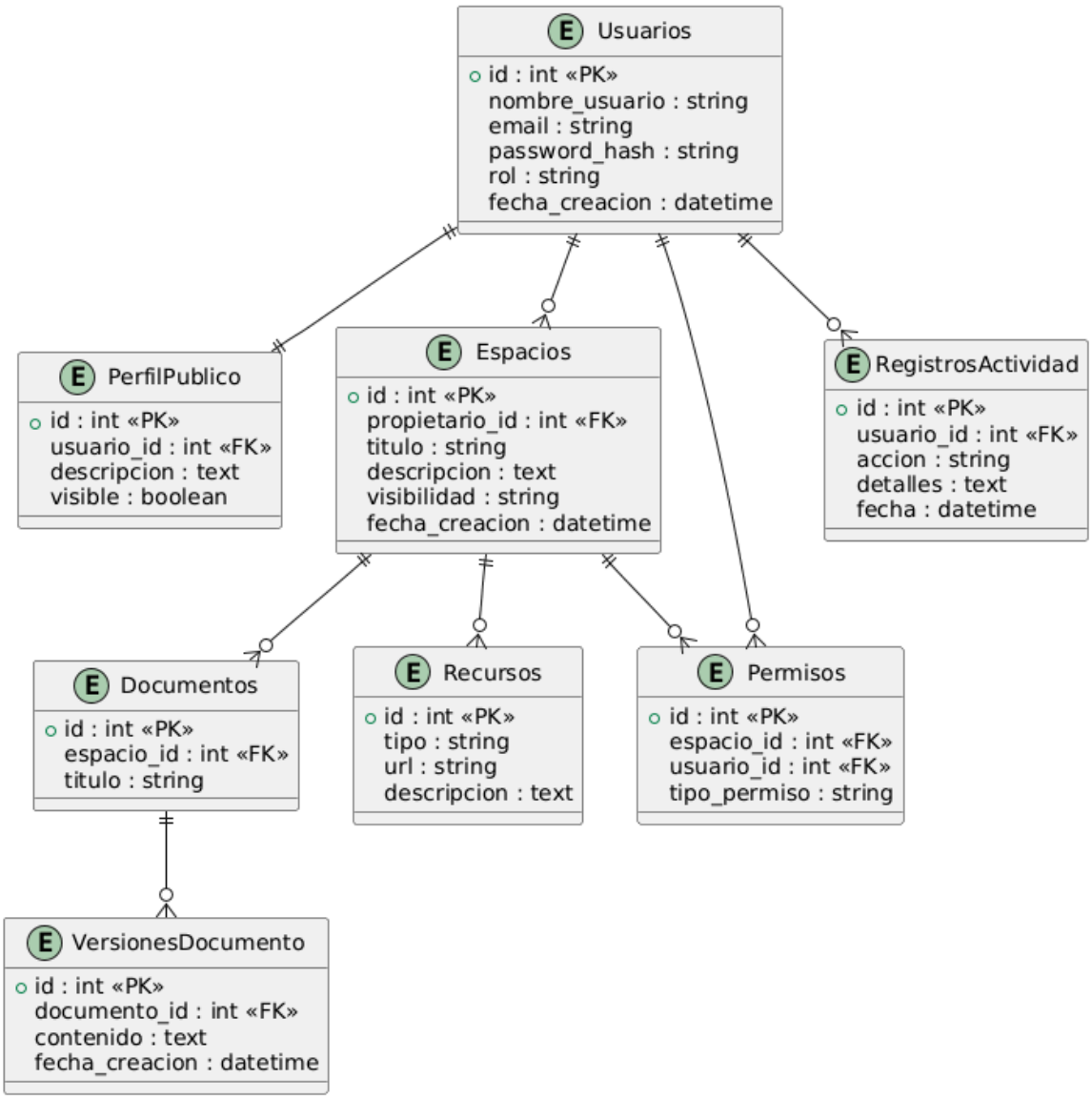
Relaciones principales:

Un usuario puede tener múltiples espacios creativos.

Cada espacio puede contener múltiples documentos.

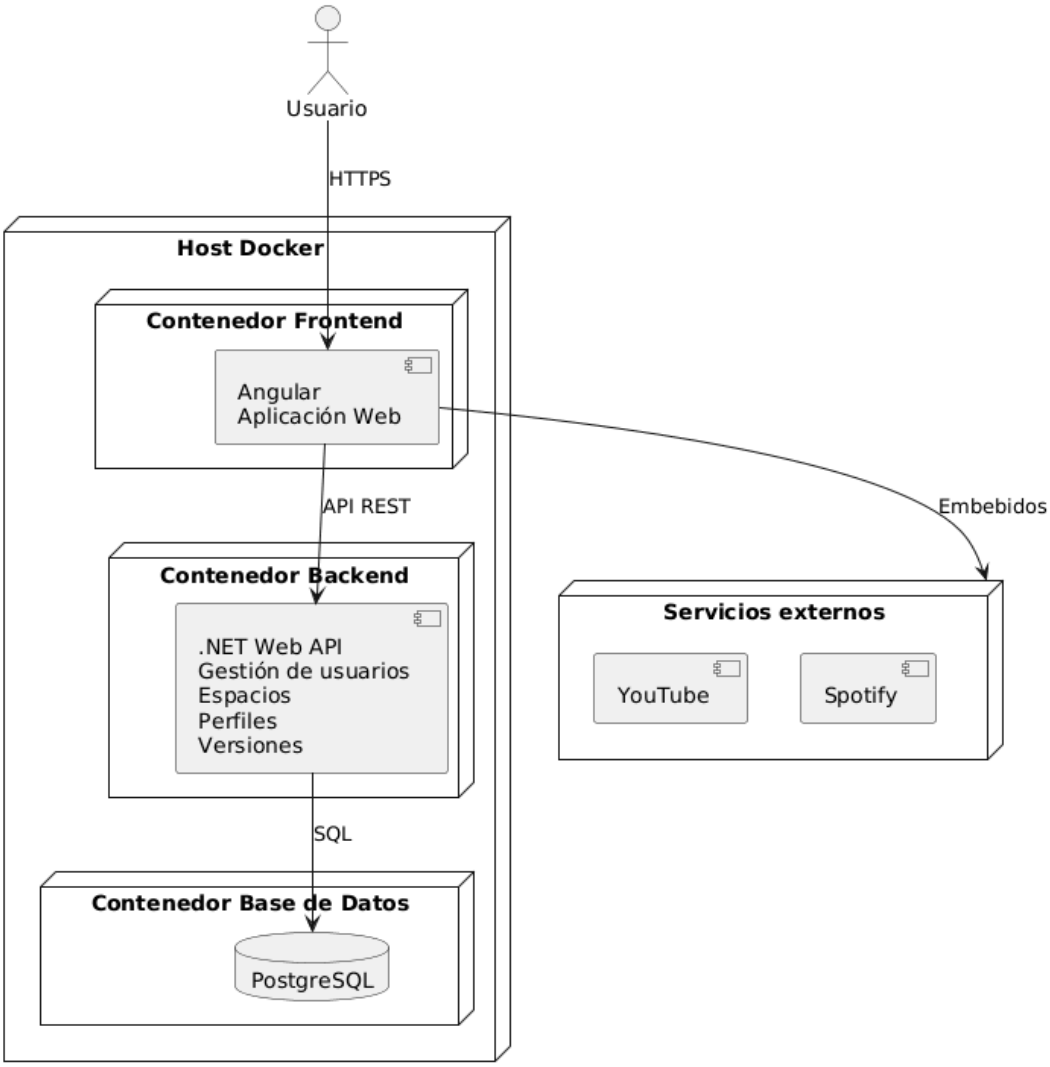
Cada documento puede tener múltiples versiones.

Los permisos permiten compartir espacios con otros usuarios.


Usuarios
id : int «PK»
nombre_usuario : string
email : string
password_hash : string
rol : string
fecha_creacion : datetime
PerfilPublico
id : int «PK»
usuario_id : int «FK»
descripcion : text
visible : boolean
Espacios
id : int «PK»
propietario_id : int «FK»
titulo : string
descripcion : text
visibilidad : string
fecha_creacion : datetime
RegistrosActividad
id : int «PK»
usuario_id : int «FK»
accion : string
detalles : text
fecha : datetime
Documentos
id : int «PK»
espacio_id : int «FK»
titulo : string
Recursos
id : int «PK»
tipo : string
url : string
descripcion : text
Permisos
id : int «PK»
espacio_id : int «FK»
usuario_id : int «FK»
tipo_permiso : string
VersionesDocumento
id : int «PK»
documento_id : int «FK»
contenido : text
fecha_creacion : datetime

Arquitectura de contenedores

El sistema se desplegará mediante Docker.



El proyecto incluirá un archivo docker-compose.yml que permitirá levantar todos los servicios del sistema mediante un único comando:

docker compose up

Opcionalmente se podrá incluir:

[SignalR] → comunicación en tiempo real

5- Planificación preliminar

Duración estimada: 8 semanas

Semana	Trabajo
1	Preparación
2-3	Backend
3	Base de datos
4-6	Frontend
7	Integración
8	Docker + pruebas

Fase 1 – Preparación (Semana 1)

Tareas:

- Definir requisitos finales
- Diseñar base de datos
- Crear repositorio Git
- Preparar entorno de desarrollo

Hito:

✔ Proyecto inicial configurado

Fase 2 – Backend (Semanas 2–3)

Tareas:

- Crear API .NET
- Implementar autenticación JWT
- Modelos de datos
- CRUD de usuarios
- CRUD de espacios creativos
- Implementar sistema de versionado de documentos

Hito:

✔ API funcional

Fase 3 – Base de datos (Semana 3)

Tareas:

- Diseño del esquema
- Migraciones
- Pruebas de persistencia

Hito:

✔ Base de datos conectada al backend

Fase 4 – Frontend (Semanas 4–6)

Tareas:

- Crear aplicación Angular
- Sistema de login
- Interfaz de espacios creativos
- Editor Markdown
- Visualización del historial de versiones
- Perfil público

Hito:

✔ Aplicación usable

Fase 5 – Integración (Semana 7)

Tareas:

- Conectar frontend y backend
- Implementar panel de administración básico
- Implementar registro de actividad del sistema
- Pruebas del flujo completo
- Corrección de errores

Hito:

✔ MVP funcional

Fase 6 – Despliegue y pruebas (Semana 8)

Tareas:

- Dockerización
- Configuración de Docker Compose
- Pruebas finales
- Documentación

Hito final:

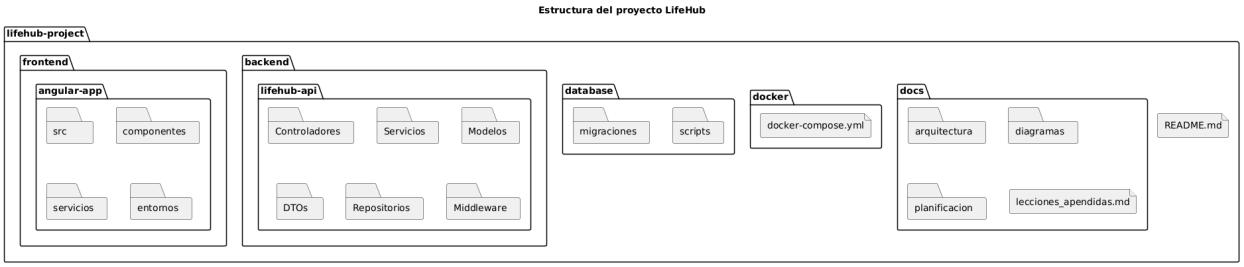
✔ Proyecto completo desplegado

Riesgos identificados

Riesgo	**Impacto**	**Mitigación**
Complejidad del editor Markdown	Medio	Usar librerías existentes
Problemas con autenticación	Alto	Implementar JWT estándar
Integración frontend-backend	Medio	Definir API clara desde el inicio
Gestión del tiempo	Medio	Priorizar funcionalidades MVP
Desarrollo de elementos fuera del MVP como la edición en tiempo real o el chat.	Alto	Desarrollo de arquitectura preparada para implementar nuevas funcionalidades a futuro después de conseguir el MVP.

6- Preparación del entorno de trabajo

Estructura proyecto



Control de versiones

El proyecto utilizará Git para el control de versiones.

Repositorio con ramas:

main

develop

feature/*

7- Bitácora

Se mantendrá este apartado donde se registrarán:

- Problemas encontrados
- Soluciones aplicadas
- Decisiones técnicas tomadas
- Mejoras futuras

Entrada 1

Fecha: 8-3-2026

Tema: elección de arquitectura

Se decide utilizar arquitectura Angular + .NET Web API por su buena integración y separación clara entre frontend y backend.